

CS 793 – MAJOR PROJECT-I
PROJECT PROGRESS REPORT

First Round Continuous Evaluation
Academic Session: 2026–2027

Project Title:

**Blockchain-Assisted Distributed AES Key Lifecycle Management
Framework for Secure Healthcare Internet of Things**

Submitted in partial fulfillment of the requirements for the course
CS 793 – Major Project-I

Submitted By

Sl. No.	Name of Student	Roll No.
1	Niraj Kumar (Team Leader)	123231203090
2	Piyush Kumar	123231203093
3	Kanhaiya Kumar	123231203069
4	Prashant Kumar	123231203094

Project Team Serial No.: 30 | Session: 9th October 2026, Forenoon (10:00 AM)

Project ID: T4-2026-030

Department of Computer Science and Engineering

B.Tech. in Computer Science and Engineering

JIS College of Engineering

Academic Session: 2026–2027



CERTIFICATE

This is to certify that the project progress report entitled “Blockchain-Assisted Distributed AES Key Lifecycle Management Framework for Secure Healthcare Internet of Things” has been prepared and submitted by the students of the B.Tech. in Computer Science and Engineering, 7th Semester, under the course CS 793 – Major Project-I.

The report presents the progress of the project work completed by the team up to the First Round Continuous Evaluation.

Project Supervisor:

Name: Dr. Bikramjit Sarkar

Designation: Professor, Department of Computer Science & Engineering

Signature: _____

Date: _____

DECLARATION

We hereby declare that the project work presented in this progress report entitled “Blockchain-Assisted Distributed AES Key Lifecycle Management Framework for Secure Healthcare Internet of Things” is the work carried out by us under the guidance of the assigned project supervisor as part of CS 793 – Major Project-I.

We further declare that the work reported here represents the progress achieved by the project team up to the First Round Continuous Evaluation.

Team Members:

Name	Roll No.	Signature
Niraj Kumar (Team Leader)	123231203090	
Piyush Kumar	123231203093	
Kanhaiya Kumar	123231203069	
Prashant Kumar	123231203094	

Date: _____

ACKNOWLEDGEMENT

We take this opportunity to express our sincere gratitude to our project supervisor, Dr. Bikramjit Sarkar, Professor, Department of Computer Science & Engineering, for his continuous guidance, valuable suggestions, and encouragement throughout the course of this project.

We are also thankful to the Department of Computer Science and Engineering and JIS College of Engineering for providing the necessary infrastructure, laboratory facilities, and academic environment that made this work possible.

We would also like to thank the Project Coordinators, Dr. Uddalak Mitra and Mr. Suprativ Saha, and the Expert Evaluation Panel for their time and constructive feedback during this evaluation, which will help us further improve our work.

Finally, we thank our fellow students, laboratory staff, and everyone who directly or indirectly supported us during the development of this project.

ABSTRACT

Healthcare Internet of Things (IoT) ecosystems continuously generate and exchange highly sensitive medical information among wearable devices, sensors, cloud servers, physicians, and healthcare institutions. While AES encryption efficiently protects this data, centralized management of the AES keys themselves introduces serious vulnerabilities — single points of failure, insider threats, unauthorized key modifications, and poor auditability. This project addresses that gap by proposing a Blockchain-Assisted Distributed AES Key Lifecycle Management Framework, in which patient records remain encrypted using AES-256 while a blockchain-enabled smart contract layer is designed to manage cryptographic key metadata, authorization policies, and lifecycle events — without ever exposing confidential medical information on-chain.

The framework is being developed using a Spring Boot and Java backend with a MySQL database, secured through JWT-based authentication and role-based access control. As of this First Round Evaluation, the team has completed and rigorously tested the entire core key-lifecycle management engine: AES-256-GCM encryption, a dedicated key protection layer, a complete key lifecycle state machine (creation, rotation, expiration, archival, and a controlled multi-step emergency recovery workflow), centralized exception handling, authentication and authorization with verified role-based access control, audit logging, and transactionally consistent database operations. All modules have been independently verified through unit tests and API-level testing, with results including a verified 401/403/200 access-control test matrix and a database-level proof of the key-rotation invariant.

Blockchain integration and the React.js frontend, which depend on this verified core, are planned for subsequent development phases and Major Project-II respectively. The expected final outcome is a complete, secure, and auditable key-management framework demonstrating measurable improvement over traditional centralized approaches in traceability, access control enforcement, and resilience against insider threats.

TABLE OF CONTENTS

Certificate.....	2
Declaration	3
Acknowledgement	4
Abstract.....	5
Table of Contents	6
List of Figures	7
List of Tables.....	8
Chapter 1: Introduction.....	9
Chapter 2: Literature Review	11
Chapter 3: Requirement Analysis.....	13
Chapter 4: Proposed Methodology	15
Chapter 5: Implementation / Development Progress.....	18
Chapter 6: Preliminary Results and Discussion	21
Chapter 7: Individual Contribution of Team Members	22
Chapter 8: Current Status of the Project.....	23
Chapter 9: Challenges and Limitations.....	24
Chapter 10: Future Work / Work Plan.....	25
Chapter 11: Conclusion	26
References	27
Appendix	28

LIST OF FIGURES

- Figure 4.1: Proposed System Architecture
- Figure 4.2: Key Lifecycle & Emergency Recovery Workflow
- Figure 8.1: Overall Project Roadmap and Current Status

LIST OF TABLES

- Table 2.1: Comparative Analysis of Existing Approaches
- Table 3.1: Hardware Requirements
- Table 3.2: Software Requirements
- Table 4.1: Database Schema Overview
- Table 5.1: Implementation Status (Phases 1–14)
- Table 6.1: Experimental Results Summary
- Table 7.1: Individual Contribution of Team Members
- Table 8.1: Project Progress Chart
- Table 10.1: Future Work Plan

CHAPTER 1: INTRODUCTION

1.1 Background

Healthcare Internet of Things (IoT) ecosystems continuously generate and exchange highly sensitive medical information among wearable devices, sensors, cloud servers, physicians, and healthcare institutions. AES provides efficient encryption for protecting patient data; however, centralized key management can introduce single points of failure, insider threats, unauthorized modifications, and poor auditability. Blockchain technology offers decentralized trust, immutability, and transparent transaction logging, making it a promising foundation for distributed cryptographic key management in this domain.

1.2 Problem Statement

The project aims to develop a blockchain-based distributed AES key lifecycle management framework for healthcare IoT ecosystems. The framework securely manages cryptographic keys throughout their lifecycle — generation, distribution, storage, periodic rotation, revocation, archival, and emergency recovery — through blockchain-enabled smart contracts. Patient records remain encrypted using AES-256, while the blockchain is intended to maintain cryptographic metadata, audit logs, authorization policies, and key lifecycle events without exposing confidential medical information.

1.3 Motivation

In a typical centralized key-management design, a single server or administrator holds authority over all encryption keys used across an entire healthcare IoT deployment. This creates a single point of failure: if that server is compromised, every patient record encrypted with keys it manages becomes vulnerable. Furthermore, centralized systems are difficult to audit reliably, since logs stored on the same server an attacker compromises can themselves be altered or deleted. Given the sensitivity of medical data and the regulatory expectations around healthcare data protection, there is a strong motivation to explore a distributed, tamper-resistant approach to key governance — one where every key operation is independently verifiable and where no single actor can unilaterally alter the historical record.

1.4 Objectives

1. To study existing centralized and blockchain-assisted key-management approaches for healthcare IoT and identify their limitations.
2. To design a layered system architecture combining AES-256 encryption for data confidentiality with a blockchain-assisted layer for key governance.
3. To develop a secure backend key-lifecycle management engine covering generation, protection, rotation, expiration, archival, and emergency recovery of AES keys.
4. To implement authentication, role-based authorization, and audit logging to ensure accountability of every key operation.
5. To evaluate the developed modules through unit testing and API-level verification, and to assess blockchain-related integration overhead in a later phase.

1.5 Scope of the Project

Included in the current scope of the project:

- A Spring Boot backend implementing the complete AES-256 key lifecycle management engine (Phases 1–14 of the development roadmap).
- JWT-based authentication and role-based access control (USER / ADMIN).
- Audit logging of all significant key-management operations.
- A relational (MySQL) data model for users, patients, encrypted medical records, and key metadata.

Outside the current scope (planned for subsequent phases / Major Project-II):

- Live blockchain network and smart contract deployment (Phases 19–23).
- The React.js frontend application, including dashboards and user-facing key-management screens (Phases 24–33).

- Integration with physical healthcare IoT hardware devices — the current prototype uses simulated IoT data sources.

1.6 Expected Outcome

The expected final outcome of this project is a working prototype that securely encrypts healthcare records using AES-256 and provides distributed, traceable, and auditable cryptographic key lifecycle management using blockchain and smart contracts, with an accessible frontend dashboard. At the current stage, the expected outcome of the backend-only development is a fully functional, independently tested key-management engine ready to receive blockchain integration.

CHAPTER 2: LITERATURE REVIEW

2.1 Existing Research/Systems

A range of prior work has explored the use of cryptography and blockchain to secure healthcare IoT data. Singh et al. (2022) surveyed blockchain-based security solutions for IoT-based e-healthcare systems, highlighting key management as one of the central open challenges in securing sensor-to-cloud data flows. Dorri et al. (2017) proposed an early blockchain-for-IoT architecture using a smart-home case study, demonstrating how a lightweight, locally-managed blockchain could remove the need for a fully centralized trusted authority. More recent work by Costa et al. (2023, IEEE Access) and Wizar et al. (2023, IEEE Access) has focused specifically on blockchain-based protocols for securing health records and on authenticated key-management mechanisms for smart healthcare applications, respectively — both directly relevant to the problem this project addresses.

2.2 Review of Relevant Literature

Several studies converge on the same underlying theme: symmetric encryption (typically AES) remains the practical choice for encrypting bulk healthcare data due to its computational efficiency on resource-constrained IoT devices, but the security of the overall system depends almost entirely on how the encryption keys themselves are managed. Zhao et al. (2017), as reviewed in a survey on blockchain applications for EHR security (PMC7362828), proposed a lightweight key-backup and recovery scheme for body sensor networks using fuzzy vault techniques to avoid storing keys directly. Other work has explored lightweight blockchain variants (e.g., using Blowfish or modified AES with chaotic S-boxes) to reduce the computational burden of combining encryption with blockchain logging on constrained devices. A common limitation noted across this literature is that many proposed systems either do not fully specify the key lifecycle (rotation, expiration, emergency recovery) or leave key-protection details (i.e., how the key itself is stored, not just the data it encrypts) underspecified.

2.3 Comparative Analysis

Existing Work/System	Method/Technology	Advantages	Limitations	Relevance to Proposed Project
Singh et al. (2022) – Blockchain solution for IoT e-healthcare	Blockchain + hybrid encryption survey	Broad coverage of blockchain-based healthcare security approaches	Survey-level; does not specify a concrete key lifecycle implementation	Motivates the need for a concrete, implementable key-lifecycle framework
Dorri et al. (2017) – Blockchain for IoT Security and Privacy	Lightweight local blockchain (smart-home case study)	Removes fully centralized trust; low overhead design	Not healthcare-specific; no AES key lifecycle detail	Informs the distributed-trust architectural approach used in this project
Costa et al. (2023) – Sec-Health, IEEE Access	Blockchain protocol for securing health records	Healthcare-specific; blockchain-backed record security	Focused on record integrity rather than key lifecycle management	Validates blockchain as suitable for healthcare audit trails, as used here
Wizar et al. (2023) – IEEE Access	Blockchain-envisioned authenticated key management for smart healthcare	Directly addresses authenticated key management in healthcare	Implementation and lifecycle depth vary; emergency recovery not emphasized	Closest related work; this project extends the idea with a full lifecycle + recovery workflow
Zhao et al. (2017), via PMC7362828 survey	Fuzzy-vault based key backup/recovery for BSNs	Avoids storing raw keys; supports recovery	Domain-specific (body sensor networks); not blockchain-integrated	Reinforces the importance of a dedicated key-protection layer, adopted in this project

2.4 Research/Development Gap

Limitation Identified	Description
-----------------------	-------------

Single point of failure	Centralized key servers remain a common design in practice, creating a single point of compromise for all protected data.
Insider threats	Privileged administrators can misuse centralized key access with limited independent oversight.
Limited auditability	Most surveyed systems do not provide a tamper-resistant, independently verifiable audit trail of key operations.
Incomplete lifecycle coverage	Few systems explicitly define and implement the full key lifecycle, including controlled emergency recovery.

2.5 Proposed Contribution

This project proposes a Blockchain-Assisted Distributed AES Key Lifecycle Management Framework that directly targets the gaps identified above by: (a) implementing a dedicated key-protection layer that never persists raw AES keys, (b) defining and implementing a complete key lifecycle state machine covering creation, rotation, time-based expiration, archival, and a separately-governed, multi-step emergency recovery workflow, (c) enforcing role-based access control with independently verified test cases, and (d) recording every key operation through an audit-logging module intended to be backed by blockchain-based tamper-resistant storage in a subsequent development phase.

CHAPTER 3: REQUIREMENT ANALYSIS

3.1 Functional Requirements

- The system shall allow user registration and authentication, always assigning the default USER role at the backend.
- The system shall generate AES-256 keys and store them only in a protected (encrypted) form.
- The system shall support key rotation, ensuring exactly one ACTIVE version exists per key at any time.
- The system shall automatically expire keys past their configured validity period using a scheduled check.
- The system shall support archival of inactive key versions while protecting the currently ACTIVE version from accidental archival.
- The system shall support a controlled emergency recovery workflow requiring explicit request, approval, and execution steps.
- The system shall enforce role-based access control, restricting administrative operations to ADMIN users.
- The system shall log every significant key-management operation with actor, action, endpoint, timestamp, and result.

3.2 Non-Functional Requirements

- Performance: Key management operations (create, rotate, retrieve metadata) should complete within acceptable API response times for interactive use.
- Security: Plaintext AES keys must never be exposed via API responses, logs, audit records, or database fields at any stage.
- Reliability: Multi-step database operations (e.g., key rotation) must be atomic, using transactional guarantees to prevent inconsistent states.
- Usability: API responses should follow a consistent, predictable structure, including standardized error responses.
- Scalability: The layered architecture (Controller → Service → Repository) should allow individual layers to be scaled or modified independently.
- Maintainability: The codebase follows a modular package structure separating configuration, security, encryption, and business logic concerns.

3.3 Hardware Requirements

Component	Minimum Requirement	Proposed Requirement
Processor	Dual-core 2.0 GHz	Quad-core 2.5 GHz or higher
RAM	4 GB	8 GB or higher
Storage	20 GB free space	50 GB free space (SSD preferred)
Other	Stable internet connection for dependency downloads and GitHub sync	Same, with a dedicated development machine per team member

3.4 Software Requirements

Software/Tool	Version	Purpose
Operating System	Windows 10/11 or Linux (Ubuntu 20.04+)	Development environment
Programming Language	Java 17+	Backend application development
Framework	Spring Boot 3.x	REST API, security, and dependency management

Database	MySQL 8.x	Persistent storage for users, patients, records, and key metadata
Other Tools	Maven, Git/GitHub, Postman, JUnit	Build management, version control, API testing, unit testing

CHAPTER 4: PROPOSED METHODOLOGY

4.1 Overall Methodology

The project follows an incremental, phase-wise development methodology. Rather than attempting to build all layers of the system (backend, blockchain, frontend) simultaneously, development was structured so that the security-critical backend core — the AES key lifecycle management engine — is built and rigorously tested first, since the blockchain integration and frontend layers both depend on this core being correct. Each phase was implemented, tested (via unit tests and Postman-based API testing), verified against expected behavior, committed to Git, and pushed to the team's GitHub repository before proceeding to the next phase.

4.2 System Architecture

The system follows a layered client-server architecture, illustrated in Figure 4.1:

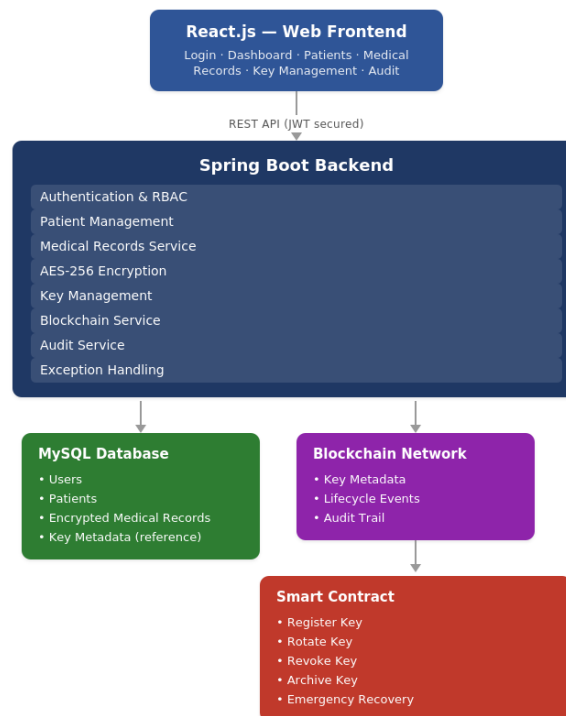


Figure 4.1: Proposed System Architecture

4.3 Workflow

A typical request flows through the system as follows: the client (frontend or API test tool) sends a REST request over HTTPS; Spring Security intercepts the request and validates the JWT bearer token; upon successful authentication, the request is authorized against the user's role; the Controller layer delegates to the Service layer, which contains the business logic (e.g., key generation, rotation); the Service layer interacts with the Repository layer for persistence via JPA/Hibernate; and finally, an Audit Logging filter records the operation before the response is returned to the client.

The key lifecycle workflow itself — covering normal operation, expiration, archival, and emergency recovery — is shown in Figure 4.2:

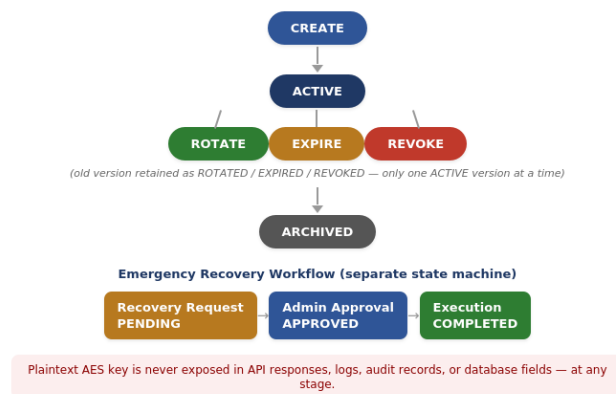


Figure 4.2: Key Lifecycle & Emergency Recovery Workflow

4.4 Modules/Components

Module 1: Authentication & Authorization

Handles user registration (always assigning the USER role by default), BCrypt password hashing, JWT-based login, and role-based access control (RBAC) enforced via Spring Security, restricting administrative endpoints to ADMIN users.

Module 2: AES Key Management & Protection

Generates AES-256 keys using Java's cryptographic APIs, encrypts data using AES/GCM/NoPadding with a random 12-byte IV, and protects generated keys using a separate master-key-based KeyProtectionService before persistence, so raw keys are never stored directly.

Module 3: Key Lifecycle Management

Implements the complete key lifecycle state machine — CREATE, ACTIVE, ROTATE, EXPIRE (via a scheduled check), REVOKE, ARCHIVE — along with a separately-governed Emergency Recovery workflow (PENDING → APPROVED → COMPLETED).

Module 4: Exception Handling & Audit Logging

Provides centralized exception handling via a GlobalExceptionHandler returning standardized error responses, and an AuditLoggingFilter that records the actor, action, endpoint, timestamp, and result of every significant operation.

4.5 Algorithms/Techniques

- AES-256 in GCM mode (AES/GCM/NoPadding) — provides both confidentiality and integrity/authenticity via an authentication tag, with a randomly generated 12-byte IV per encryption operation to prevent ciphertext pattern repetition.
- BCrypt password hashing — one-way adaptive hashing for user credentials, never storing plaintext passwords.
- JSON Web Tokens (JWT) — stateless authentication carrying the user's identity and role as claims, validated on every request via a custom JwtAuthenticationFilter.
- Finite state machines — used to model both the KeyStatus lifecycle (ACTIVE / ROTATED / EXPIRED / REVOKED / ARCHIVED) and the independent RecoveryStatus lifecycle (PENDING / APPROVED / REJECTED / COMPLETED), each with explicitly validated transitions.
- Database transactions (@Transactional, ACID properties) — used to guarantee that multi-step lifecycle operations such as key rotation apply atomically.

4.6 Database/Data Design

The current relational schema (MySQL, via JPA/Hibernate) is summarized below:

Table	Key Fields	Purpose
-------	------------	---------

users	id, name, email, password, role, created_at	Stores registered users and their assigned role (USER/ADMIN).
patients	id, user_id, patient_code, date_of_birth, created_at	Stores patient records linked to a user account.
medical_records	id, patient_id, encrypted_data, key_id, key_version, created_at, updated_at	Stores AES-256 encrypted medical data, referencing the key used.
key_metadata	id, key_identifier, version, status, algorithm, created_at, rotated_at, revoked_at	Stores key lifecycle metadata (never the raw key).
key_lifecycle_events / audit_logs	id, key_id / username, operation / action, performed_by, timestamp, status, blockchain_tx_hash	Records every key operation for traceability and audit.
key_recovery_requests	id, key_id, status (PENDING/APPROVED/COMPLETED), requested_by, approved_by, timestamps	Tracks the emergency recovery workflow independently of key status.

CHAPTER 5: IMPLEMENTATION / DEVELOPMENT PROGRESS

5.1 Development Environment

- Backend: Java 17, Spring Boot 3.x, Spring Security, Spring Data JPA/Hibernate
- Database: MySQL 8.x (local instance, database: healthcare_key_management)
- Build Tool: Maven
- Version Control: Git, with the repository hosted on GitHub
- API Testing: Postman
- Unit/Integration Testing: JUnit
- Secrets Management: Environment-based configuration (DB credentials, JWT secret, AES master key supplied via environment variables, never hard-coded)

5.2 Implementation Status

The table below summarizes all completed backend phases as of this First Round Evaluation:

Phase	Module	Status	Summary
1	Project Foundation	● Completed	Spring Boot + Java project structure, layered architecture (Controller → Service → Repository).
2	Core Setup	● Completed	Maven dependencies, application configuration, MySQL connectivity via JPA/Hibernate.
3	Key Management (Base)	● Completed	AES-256 key generation, key metadata model (keyId, version, status, algorithm).
3.1 / 3.2	Extended Key Functionality	● Completed	AES-GCM mode with random IV; response DTOs excluding sensitive key material.
5	Security Foundation	● Completed	AES-256-GCM encryption verified — confidentiality + integrity via authentication tag.
6	Key Protection	● Completed	Master-key-based protection layer (KeyProtectionService) — plaintext AES keys never stored directly.
7	Key Lifecycle	● Completed	Full state machine: CREATE → ACTIVE → ROTATE / REVOKE, with versioning.
8	Exception Architecture	● Completed	Centralized GlobalExceptionHandler, custom exceptions, standardized ErrorResponse.
9	Authentication & Authorization	● Completed	JWT-based login, BCrypt hashing, role-based access control (USER / ADMIN).
10	Audit Logging	● Completed	AuditLoggingFilter records who/what/when/status for every request.
11	Key Expiration	● Completed	Scheduler-based automatic expiry check (60s interval) using expiresAt field.
12	Key Archival	● Completed	Inactive key versions archived instead of deleted; active version protected.
13	Emergency Recovery	● Completed	Controlled PENDING → APPROVED → COMPLETED recovery workflow.
14	DB Transactions	● Completed	@Transactional-backed atomic operations verified via key rotation test.

Overall: 14 out of the full 33-phase roadmap are complete — covering the entire backend security core. All code has been committed and pushed to the team's GitHub repository.

5.3 Implementation Details

AES-256-GCM Encryption

Encryption uses AES/GCM/NoPadding with a 256-bit key and a random 12-byte Initialization Vector (IV) generated per operation. GCM mode provides both confidentiality and integrity — any tampering with the ciphertext is detectable via the authentication tag.

Plaintext + AES-256 Key + Random 12-byte IV → AES-GCM → Ciphertext + Authentication Tag

Key Protection Layer

Rather than storing generated AES keys directly, each key is passed through a KeyProtectionService that encrypts it using a separate master key (supplied via environment configuration) before persistence. API responses (via KeyMetadataResponse DTOs) intentionally exclude both the protected and plaintext key material.

Authentication & Authorization

User registration always assigns the default USER role at the backend — clients cannot self-assign ADMIN. On successful login, a JWT is issued containing the user's email and role as claims. A JwtAuthenticationFilter validates the token on each request. The following access-control behavior was explicitly tested on a protected admin endpoint:

Scenario	Request	Result
No JWT Token	GET /api/admin/check	401 Unauthorized — no authentication present
USER JWT Token	GET /api/admin/check	403 Forbidden — authenticated but not ADMIN role
ADMIN JWT Token	GET /api/admin/check	200 OK — admin access granted

Exception Handling

All exceptions are handled centrally via a @RestControllerAdvice-based GlobalExceptionHandler, returning a consistent ErrorResponse structure (timestamp, status, error, message, path) instead of raw stack traces.

Audit Logging

An AuditLoggingFilter runs after JWT authentication, so the authenticated user's identity is available for each recorded action. Every significant operation is persisted with actor, action, endpoint, timestamp, and result.

Database Transactions

Multi-step key lifecycle operations (such as rotation) are wrapped in @Transactional boundaries to guarantee atomicity. The following test was performed to verify this:

Test Step	Request / Action	Result
1. Create key	POST /api/keys — keyId: PHASE14-TX-001	id=21, version=1, status=ACTIVE
2. Rotate key	PATCH /api/keys/PHASE14-TX-001/rotate	id=22, version=2, status=ACTIVE
3. Verify DB state	Query all versions for PHASE14-TX-001	v1 → ROTATED, v2 → ACTIVE
4. Verify invariant	Count ACTIVE versions for keyId	Active count = 1 (confirmed)

5.4 User Interface / Prototype

As of this First Round Evaluation, development has been focused entirely on the backend security core, and the React.js frontend has not yet been started (it is scheduled for Phases 24–33, in Major Project-II). All functionality described in this report has therefore been verified at the API level using Postman, rather than through a graphical interface. No UI screenshots are included in this report for that reason; API request/response evidence is presented instead in Section 6.2.

5.5 Current Functionalities

The following functionalities have been successfully implemented and verified as of this evaluation:

- User registration and login with JWT issuance
- Role-based access control distinguishing USER and ADMIN privileges
- AES-256-GCM based encryption and decryption of data
- Secure key generation with master-key-based protection before storage
- Key creation, rotation, and versioning with a single-ACTIVE-version invariant
- Scheduler-driven automatic key expiration
- Key archival with protection against archiving an ACTIVE version
- A controlled, multi-step emergency key recovery workflow
- Centralized exception handling with standardized error responses
- Audit logging of key-management operations
- Transactionally consistent multi-step database operations

CHAPTER 6: PRELIMINARY RESULTS AND DISCUSSION

6.1 Results Obtained So Far

Test Area	Test Performed	Outcome
Encryption	AES-256-GCM encrypt/decrypt round-trip with random IV	Passed — plaintext correctly recovered; ciphertext differs per encryption
Authorization	Admin endpoint access with No JWT / USER JWT / ADMIN JWT	Passed — 401 / 403 / 200 responses exactly as expected
Key Rotation	Rotate active key and verify database state	Passed — old version ROTATED, new version ACTIVE, only 1 active count
Key Expiration	Scheduler-based expiry check on expiresAt	Passed — expired keys automatically transitioned to EXPIRED
Key Archival	Attempt to archive with an ACTIVE version present	Passed — system correctly blocks archival of active version
Emergency Recovery	Invalid state transitions (e.g., execute before approve)	Passed — invalid transitions rejected with conflict response
Exception Handling	Request for a non-existent key	Passed — returns structured 404 response, not a stack trace

6.2 Testing Performed

Testing was carried out at two levels. First, unit tests (JUnit) were used to verify the correctness of the AES-256-GCM encryption/decryption routines in isolation. Second, API-level testing (Postman) was used to verify end-to-end behavior of authentication, authorization, and key-lifecycle endpoints, including deliberately testing invalid and unauthorized requests to confirm the system fails safely and predictably.

6.3 Performance/Accuracy Evaluation

At this stage of development, evaluation has focused on functional correctness rather than formal performance benchmarking, since the goal of this phase was to establish a correct and secure key-management core before optimizing for scale. All 7 test areas listed in Section 6.1 passed on both expected-success and expected-failure paths (a 100% pass rate across the tests performed to date). Formal performance metrics — such as average API response time under load and blockchain write latency — are planned once blockchain integration begins (Phase 19 onward), as blockchain overhead is one of the project's explicit research objectives.

6.4 Discussion

The results obtained so far confirm that the core security invariants the project set out to guarantee are holding under actual execution: exactly one ACTIVE key version exists per keyId after rotation, unauthorized users cannot access administrative key operations, and the system fails predictably (with structured error responses) rather than crashing on invalid input. These results directly address several of the limitations identified in Chapter 2 — notably, the lack of auditability and weak access control common in centralized designs are already measurably improved through the audit logging and RBAC modules.

The main current limitation is that the blockchain layer, which is intended to make the audit trail itself tamper-resistant (rather than just present), has not yet been integrated — audit logs currently reside in the same relational database as other application data. This is a deliberate, sequenced choice explained further in Chapter 9, and is the primary focus of the next development stage.

CHAPTER 7: INDIVIDUAL CONTRIBUTION OF TEAM MEMBERS

Each team member's primary area of responsibility and completed work is summarized below. All team members remain familiar with the complete project and are prepared to answer questions on any module during the evaluation.

Student Name	Roll No.	Major Responsibilities	Work Completed
Niraj Kumar (Team Leader)	123231203090	Overall system architecture and backend development	Designed the layered system architecture; led backend development and integration across all modules (Phases 1–14); coordinated team progress and documentation.
Piyush Kumar	123231203093	Authentication & Authorization	Implemented user registration, BCrypt password hashing, JWT-based login, JWT authentication filter, and role-based access control; verified the 401/403/200 access-control test matrix.
Kanhaiya Kumar	123231203069	Database Design & Handling	Designed the relational database schema (users, patients, medical records, key metadata, audit logs); implemented JPA/Hibernate entities and repositories; supported transactional consistency testing.
Prashant Kumar	123231203094	Blockchain Integration (Upcoming Phase)	Researching blockchain architecture, private/local network options, and smart contract design in preparation for Phases 19–23; supporting integration planning between the backend and the blockchain layer.

CHAPTER 8: CURRENT STATUS OF THE PROJECT

The project follows a 33-phase development roadmap spanning backend development, blockchain integration, and frontend development. As of this First Round Evaluation, all 14 phases planned for the backend security core are complete, verified, and pushed to GitHub. Figure 8.1 shows the complete roadmap and the team's current position within it.

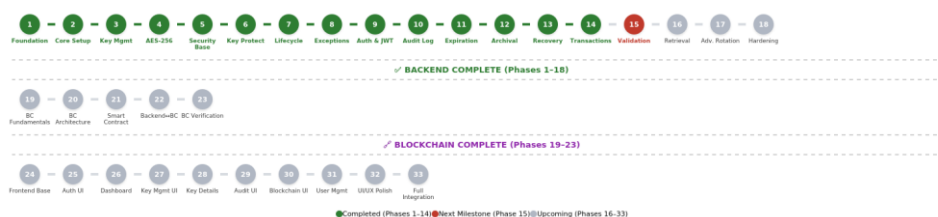


Figure 8.1: Overall Project Roadmap and Current Status

Project Progress Chart

Activity	Planned	Completed	Percentage/Status
Problem Definition	Yes	Yes	100% — Completed
Literature Review	Yes	Yes	100% — Completed (this report)
Requirement Analysis	Yes	Yes	100% — Completed
System Design	Yes	Yes	100% — Completed
Module Development (Backend Core)	Yes	Yes	100% — Phases 1–14 Completed
Implementation (Blockchain)	Yes	No	0% — Planned (Phases 19–23)
Implementation (Frontend)	Yes	No	0% — Planned for Major Project-II
Testing (Backend)	Yes	Yes	Completed for all implemented modules
Documentation	Yes	Yes	Ongoing — this report + supporting technical documents

CHAPTER 9: CHALLENGES AND LIMITATIONS

Several technical challenges were encountered during backend development, along with the measures taken to address them:

- Ensuring exactly one ACTIVE key version per keyId during rotation: solved by wrapping the rotate operation in a @Transactional boundary and explicitly verifying the active-count invariant through database-level testing.
- Preventing accidental exposure of key material: solved by designing dedicated response DTOs (e.g., KeyMetadataResponse) that structurally exclude protected/plaintext key fields, rather than relying on manual filtering at each endpoint.
- Coordinating JWT authentication with audit logging: solved by ordering the AuditLoggingFilter after the JWT authentication filter in the security filter chain, so the authenticated identity is available when the audit record is created.
- Designing safe emergency recovery: solved by introducing an independent RecoveryStatus state machine (separate from KeyStatus) with explicit request → approval → execution steps, rather than allowing direct reactivation of an archived key.
- Scheduling automatic key expiration without manual polling: solved using a Spring-managed scheduled task that periodically checks for keys past their expiresAt timestamp.
- Time management across four team members working on interdependent modules: addressed through phase-wise development, where each phase was completed, tested, and committed before the next began, reducing integration conflicts.

The primary current limitation of the project is that the blockchain layer and frontend application have not yet been developed, as described in Chapter 1 (Scope). This was a deliberate sequencing decision — both layers depend on the correctness of the backend key-management engine, which has therefore been prioritized and thoroughly verified first.

CHAPTER 10: FUTURE WORK / WORK PLAN

The following activities are planned to complete the project, building on the verified backend core:

Stage	Phases	Focus
Before Second Round Evaluation	15 – 18	Validation, secure key retrieval hardening, advanced rotation, API documentation (Swagger/OpenAPI), testing, and backend hardening — completing the backend.
Subsequent Evaluations (Sem-7)	19 – 23	Blockchain fundamentals, blockchain architecture design, smart contract development, backend↔blockchain integration, and blockchain verification.
Major Project-II (Sem-8)	24 – 33	Full React.js frontend (authentication UI, dashboard, key management, key details, audit UI, blockchain UI), user management, UI/UX polish, and full end-to-end system integration.

Planned Activity	Expected Completion	Responsible Member(s)
Validation & backend hardening (Phases 15–18)	Before Second Round Evaluation	Niraj Kumar, Piyush Kumar, Kanhaiya Kumar
Blockchain network setup & smart contract design (Phases 19–21)	Subsequent evaluation cycle	Prashant Kumar (lead), Niraj Kumar
Backend ↔ Blockchain integration & verification (Phases 22–23)	Subsequent evaluation cycle	Prashant Kumar, Niraj Kumar
Frontend development & full integration (Phases 24–33)	Major Project-II (Sem-8)	All team members

This addresses how the current prototype will be converted into the final project: the verified backend engine will next be connected to a blockchain network so that key lifecycle events become tamper-resistant, followed by development of the frontend that will let end users (Admins, Doctors, Patients, Auditors) interact with the complete system.

CHAPTER 11: CONCLUSION

This report has presented the progress of the project “Blockchain-Assisted Distributed AES Key Lifecycle Management Framework for Secure Healthcare Internet of Things” as of the First Round Continuous Evaluation. The problem addressed — the security and auditability weaknesses of centralized AES key management in healthcare IoT — was motivated through background research and a comparative review of existing blockchain-assisted healthcare security literature.

The team has completed and independently verified the entire backend key-lifecycle management engine, comprising 14 development phases: project foundation, AES-256-GCM encryption, a dedicated key protection layer, a complete key lifecycle state machine (including expiration, archival, and a controlled emergency recovery workflow), centralized exception handling, JWT-based authentication with verified role-based access control, audit logging, and transactionally consistent database operations. All modules were tested through unit tests and API-level verification, with results including a fully verified access-control test matrix and a database-level proof of the key-rotation consistency invariant.

The remaining work — backend hardening and validation, blockchain integration, and frontend development — is clearly scoped and scheduled across the remaining phases of the roadmap, extending into Major Project-II. The expected final outcome remains a complete, secure, distributed, and auditable key-management framework for healthcare IoT, and the team is confident that the verified core presented in this report provides a solid foundation for that outcome.

REFERENCES

- [1] A. Singh, K. Chatterjee, and S. C. Satapathy, "Security of IoT-Based E-Healthcare System: A Blockchain Solution," ResearchGate preprint/publication, 2022.
- [2] A. Dorri, S. S. Kanhere, R. Jurdak, and P. Gauravaram, "Blockchain for IoT Security and Privacy: The Case Study of a Smart Home," in Proc. IEEE International Conference on Pervasive Computing and Communications Workshops (PerCom Workshops), 2017.
- [3] L. D. Costa et al., "Sec-Health: A Blockchain-based Protocol for Securing Health Records," IEEE Access, vol. 11, 2023, doi: 10.1109/ACCESS.2023.3245046.
- [4] M. Wizar et al., "Design of Robust Blockchain-Envisioned Authenticated Key Management Mechanism for Smart Healthcare Applications," IEEE Access, vol. 11, 2023, doi: 10.1109/ACCESS.2023.3310264.
- [5] "Applications of Blockchain in Ensuring the Security and Privacy of Electronic Health Record Systems: A Survey," PMC7362828, National Center for Biotechnology Information.
- [6] "A Lightweight Blockchain to Secure Data Communication in IoT Network on Healthcare System," International Journal of Safety and Security Engineering (IIETA), vol. 13, no. 6.
- [7] Oracle Corporation, "Java Cryptography Architecture (JCA) Reference Guide," Java SE Documentation.
- [8] Spring Framework Documentation, "Spring Security Reference," VMware/Spring.io, accessed 2026.

APPENDIX

The following supporting material is available and can be produced on request during the evaluation:

- Complete database schema (DDL) for all tables listed in Section 4.6
- Postman collection covering all tested API endpoints (authentication, key management, admin operations)
- Key source code excerpts: KeyProtectionService, JwtAuthenticationFilter, GlobalExceptionHandler, AuditLoggingFilter
- GitHub repository commit history corresponding to Phases 1–14
- Detailed phase-wise technical notes prepared internally by the team during development